

Module 18: Concurrency Basics

Two things go wrong when threads share data, and they are not the same thing.

Races are about interleaving: two threads interfere mid-operation. **Visibility** is about one thread never seeing another's write at all. This module demonstrates both, then covers the tools that fix each.

What You'll Learn

- Threads, interruption, and why you should not create threads yourself
- Why `count++` loses increments, shown with hundreds of thousands lost
- `synchronized`, atomics and `LongAdder`, and what each costs
- The visibility problem, where a loop never exits
- `ExecutorService`, and the failure mode that stays silent

Coming From Another Language

	Python	Java
Real parallelism	blocked by the GIL	yes, genuinely
Race on <code>x += 1</code>	possible	certain , at scale
Lock	<code>threading.Lock</code>	<code>synchronized</code> , <code>ReentrantLock</code>
Atomic counter	no built-in	<code>AtomicInteger</code>
Thread pool	<code>ThreadPoolExecutor</code>	<code>ExecutorService</code>
Visibility keyword	none needed	<code>volatile</code>

Python's global interpreter lock hides most of this. Java runs threads on real cores in parallel, so every hazard here is one you will actually meet.

The Lesson

Threads directly

```
Thread worker = new Thread(() -> doWork(), "worker-1");
worker.start();
worker.join();
```

`start()` and `run()` are not the same. `start()` creates a thread; `run()` just calls a method on the current one. The mistake looks like it works because the output is identical when there is nothing to

interleave.

Interruption is a request, not a kill. Catching `InterruptedException` clears the flag, so restore it:

```
catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}
```

`Thread.stop()` was deprecated in 2000 and has been removed, because killing a thread mid-operation leaves whatever it was mutating in an unknown state.

Why not to create threads yourself: a platform thread costs about 1MB of stack, there is no bound on how many exist, and nothing collects results or propagates failures.

The race

From `TheRaceCondition.java`, four threads incrementing 200,000 times each:

```
expected total: 800000  
actual total:   242657  
-> 557343 increments were LOST
```

`count++` is three operations: read, add, write. Two threads can both read `100` before either writes `101`, and one increment vanishes.

The same shape appears everywhere: `x += 1`, `list.add(item)`, `balance -= amount`, and every check-then-act pair:

```
if (!map.containsKey(k)) { map.put(k, v); }
```

Both threads can pass the check before either puts. That is why `Map` has `putIfAbsent` as a single atomic operation.

Three fixes

From `FixingTheRace.java`:

unsafe ++	796131	WRONG
synchronized	800000	correct
AtomicInteger	800000	correct
LongAdder	800000	correct

`synchronized` gives mutual exclusion *and* a happens-before edge: everything a thread did before releasing the lock is visible to the next thread that acquires it. **That second guarantee is the one people forget**, and it is why `synchronized` fixes visibility too. The cost is that blocked threads queue.

Atomics use a compare-and-swap instruction: write back only if nothing changed, otherwise retry. No blocking, but under contention the retries are the cost.

`LongAdder` keeps several internal cells so threads mostly do not touch the same memory. Faster than `AtomicLong` under heavy write contention, slower to read. Use it for metrics; use `AtomicLong` when you need the value after every update.

And the fourth option, usually best: do not share the state. Module 14 made the same point about parallel streams, where the fix was `collect` giving each thread its own container. A race you cannot have is cheaper than one you synchronise.

What not to lock on:

Don't	Because
<code>synchronized (this)</code>	anyone with a reference can lock you out
<code>synchronized (SomeClass.class)</code>	the same, globally
a <code>String</code> literal	literals are pooled and shared with unrelated code
a boxed <code>Integer</code>	values below 128 are cached

Use a `private final Object`.

Visibility

This is the one people do not expect. From `MemoryVisibility.java`, one thread spins on a flag and another sets it:

```
main set plainFlag = true
after waiting 1.5s, reader still running: true

main set volatileFlag = true
after waiting 1.5s, reader still running: false
```

The plain reader **never saw the write**. There is no interleaving here at all.

Nothing in that loop touches shared state, so the JIT may assume `plainFlag` cannot change and hoist the read out entirely, turning it into `while (true)`. That is a **legal** optimisation. The Java Memory Model only promises one thread sees another's write when a happens-before relationship connects them, and a plain field creates none.

What `volatile` does: every read goes to main memory, and it establishes happens-before, so everything written before a volatile write is visible after the matching read.

What it does not do: make `count++` atomic. Three operations remain three operations.

`volatile` is for flags and for publishing a reference. `synchronized`, atomics and locks are for compound actions.

ExecutorService

```
try (ExecutorService pool = Executors.newFixedThreadPool(3)) {
    Future<Integer> result = pool.submit(() -> compute());
    System.out.println(result.get());
}
```

`ExecutorService` has been `AutoCloseable` since Java 19, so `try-with-resources` replaces the `shutdown` and `awaitTermination` pair people routinely got wrong.

A real trap, from `UsingExecutors.java`:

```
submitted, and nothing has been reported yet
get() threw ExecutionException
caused by: java.lang.IllegalStateException: the task failed
```

A task submitted with `submit()` that throws and whose `Future` is never inspected **fails silently**. Nothing logs, nothing crashes. Use `execute()` if you want the default handler to report it, or always inspect the `Future`.

Pool	Use
<code>newFixedThreadPool(n)</code>	bounded; the safe default for CPU work
<code>newCachedThreadPool()</code>	unbounded; a burst of tasks makes a burst of threads
<code>newSingleThreadExecutor()</code>	one at a time, in order
<code>newScheduledThreadPool(n)</code>	delayed and repeating

For IO-bound work all of these force a choice between too few threads and too many. Module 19 removes the choice.

Run It

```
java modules/18-concurrency-basics/examples/ThreadsAndRunnables.java
java modules/18-concurrency-basics/examples/TheRaceCondition.java
java modules/18-concurrency-basics/examples/FixingTheRace.java
java modules/18-concurrency-basics/examples/MemoryVisibility.java
java modules/18-concurrency-basics/examples/UsingExecutors.java

./scripts/verify-examples.sh modules/18-concurrency-basics
```

Run the race example several times. The number lost changes every run, and it is never zero on a multi-core machine. That inconsistency is the lesson.

Common Mistakes

Calling `run()` instead of `start()`. No thread is created.

Assuming `volatile` makes `++` safe. It does not.

Swallowing `InterruptedException`. Restore the flag or the shutdown signal is lost.

Locking on `this`, a class, a `String` or a boxed `Integer`. All are shared more widely than you think.

Not checking a `Future`. Exceptions disappear entirely.

`newCachedThreadPool` in a server. Unbounded thread creation under load.

Check-then-act on a concurrent map. `containsKey` then `put` is two operations. Use `putIfAbsent` or `compute`.

Synchronising everything to be safe. Contention can make the parallel version slower than the sequential one, and Module 14 showed exactly that.

Key Takeaways

- `count++` is three operations, so it loses increments under contention.
- `synchronized` gives mutual exclusion and visibility. Atomics give lock-free updates. `LongAdder` wins under heavy write contention.
- Visibility is a separate hazard. A plain field can be cached or hoisted, and the reader may never see the write.
- `volatile` fixes visibility, not atomicity.
- Prefer not sharing state at all over synchronising access to it.
- Use an `ExecutorService`, close it with `try-with-resources`, and always inspect the `Future`.

Homework

[homework/README.md](#)

Build a counter four ways, break it, then fix a check-then-act race. Reference solution in [solutions/18-concurrency-basics/](#).